

Agenda.

→ Intro to VCS.

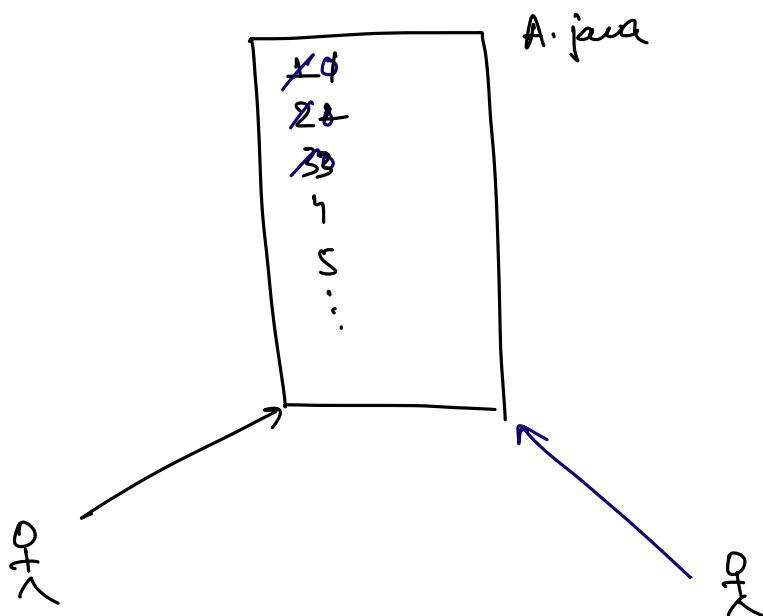
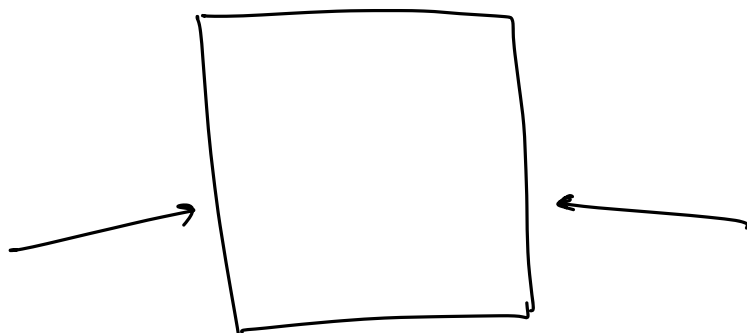
→ Centralized (Vc) Distributed VCS.

→ GIT.

Intro to VCS.

↳ Codebase.

⇒ Multiple people are going to work on a codebase at a time.

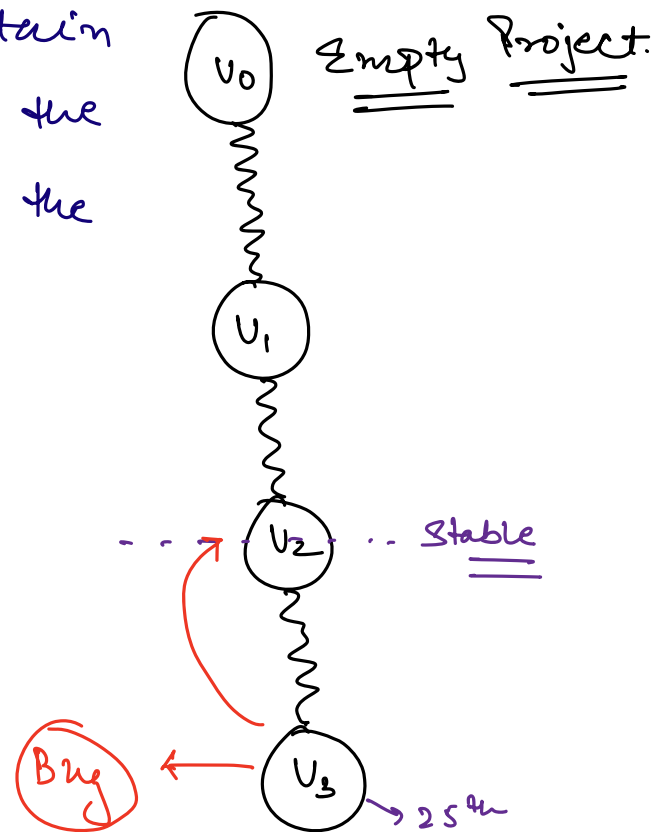


- 1) Conflict happens when multiple developers are working on the codebase.
- 2) Code history.
- 3) We might want to check the exact history of the codebase.

Version Control System

⇒ VCS allows us to maintain the complete history of the codebase by storing all the versions.

⇒ Each version contains meaningful amount of code changes on top of previous version.



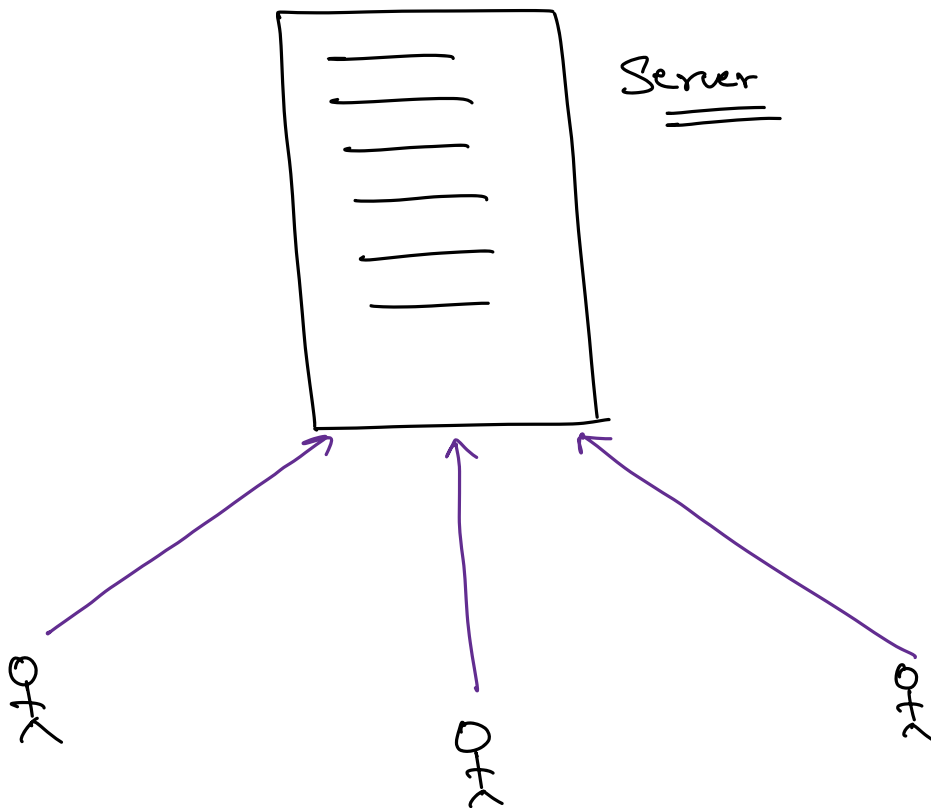
⇒ If there's any Bug in the current version of the codebase, then we need to rollback to the previous stable version.

Types of VCS.

- Centralized VCS
- Distributed VCS.

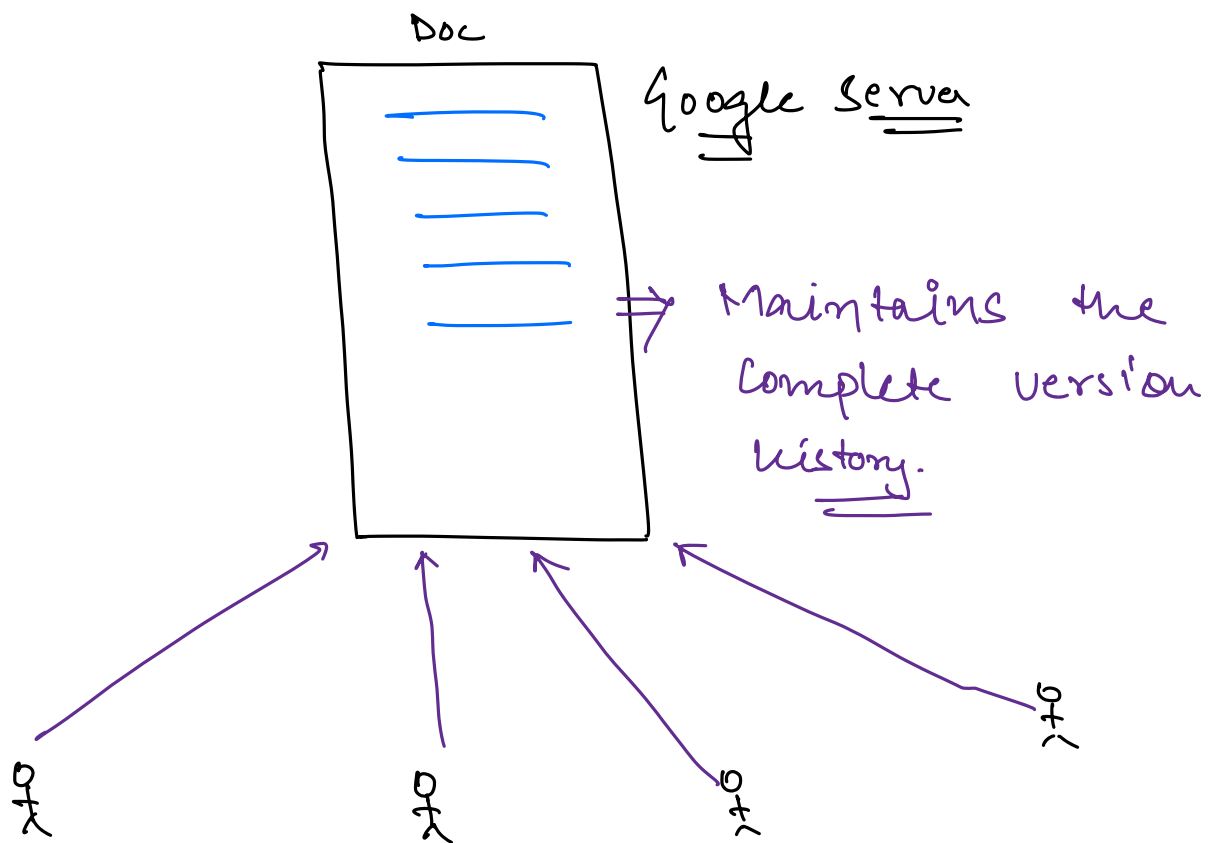
Centralized VCS.

- Complete codebase is present on a centralized server.



⇒ We need to be connected to the server in order to make code changes.

Google Doc.



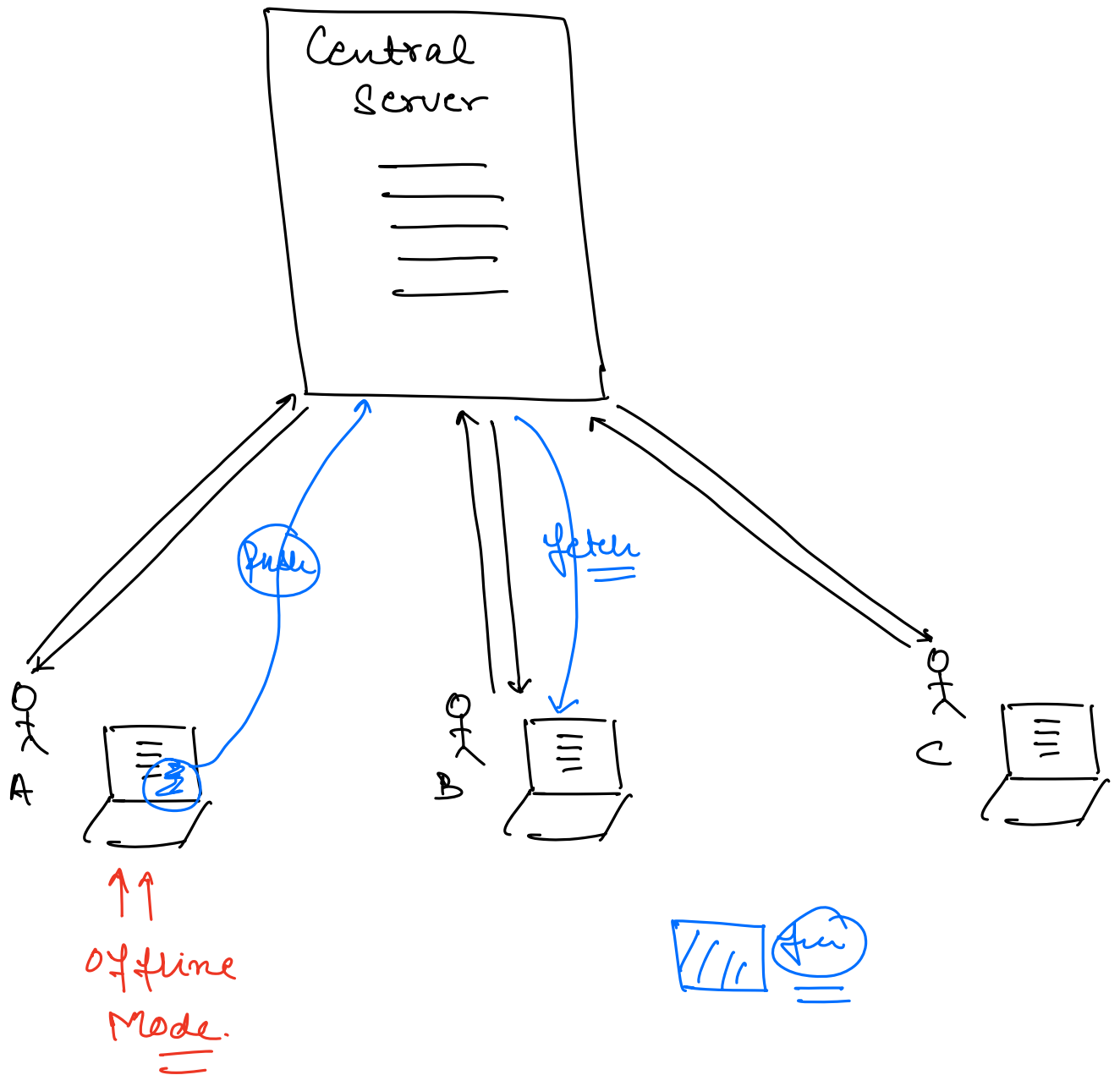
Cons.

1. SPOF (Single Point of Failure)
2. We need to be connected to the internet in order to make any changes.

Pros.

1. Security.
2. Simple
3. Space efficient.

Distributed VCS.



- 1) User downloads the codebase from the Central Server in their local mlc.
↳ CLONING. (Along with the complete code history.)

2. Users can make changes in the codebase in their local m/c without connecting to the central server.

3. Once user is done with the code changes, user needs to submit the code changes to the central server.

↳ N/w is required.
↳ Push operation

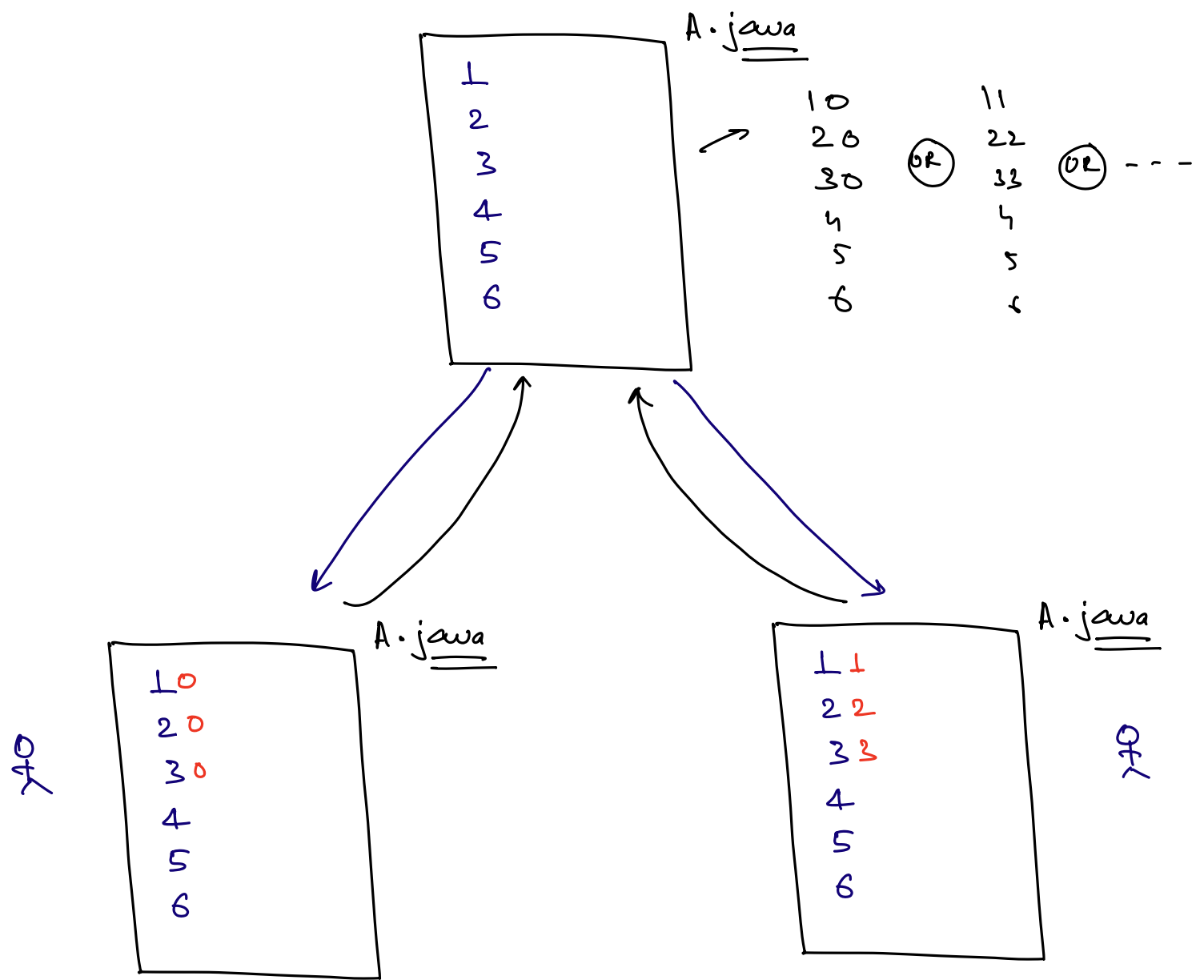
Pros.

→ No need to be online to work on the codebase

→ No SPOF.

Cons.

→ Merge Conflict



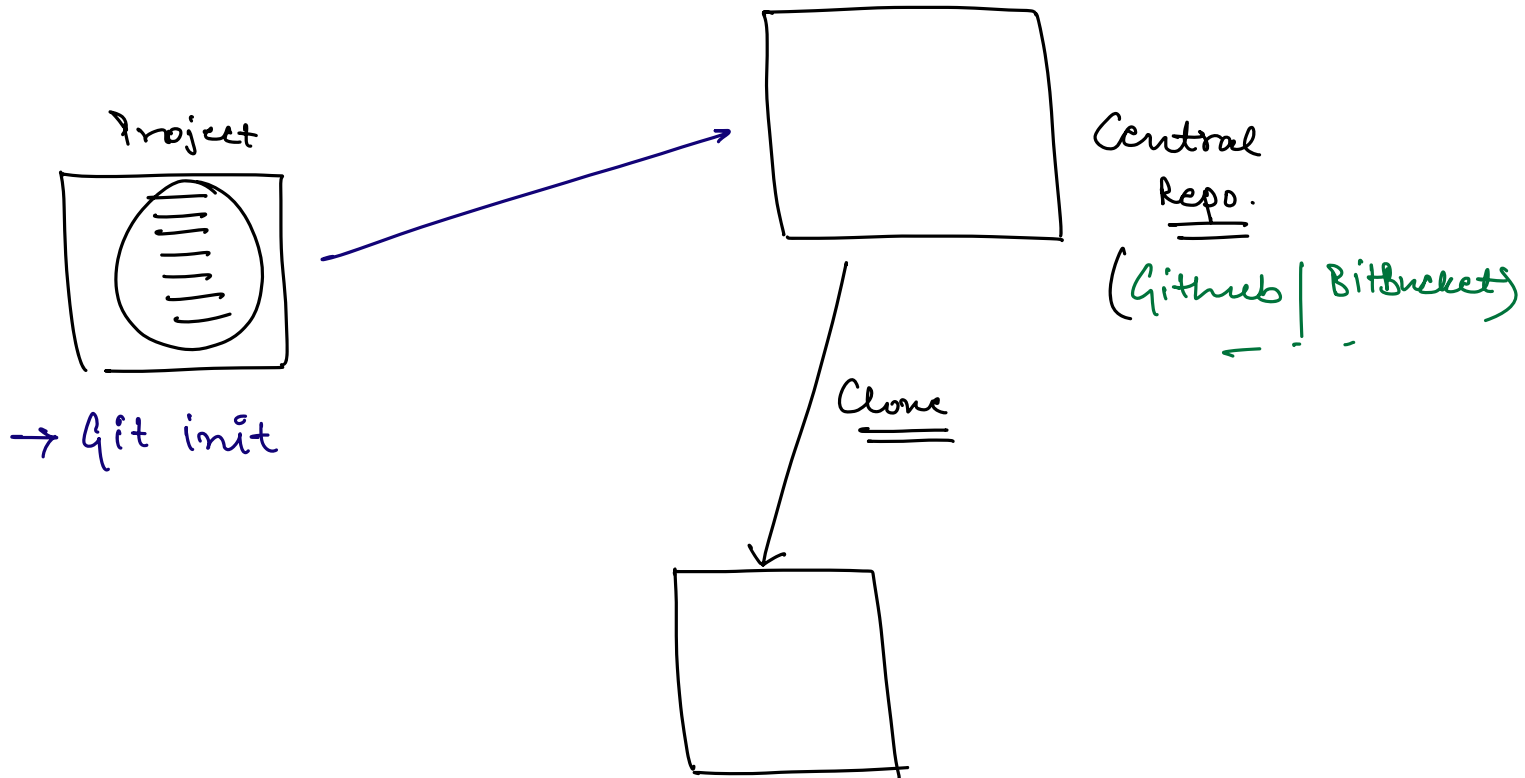
⇒ Ambiguity.

Git : Most popular distributed VCS, using which we can maintain the complete history of the codebase and multiple developers can work together.

was created by the same person who created Linux OS.

⇒ Github | BitBucket | Gitlab - . . .

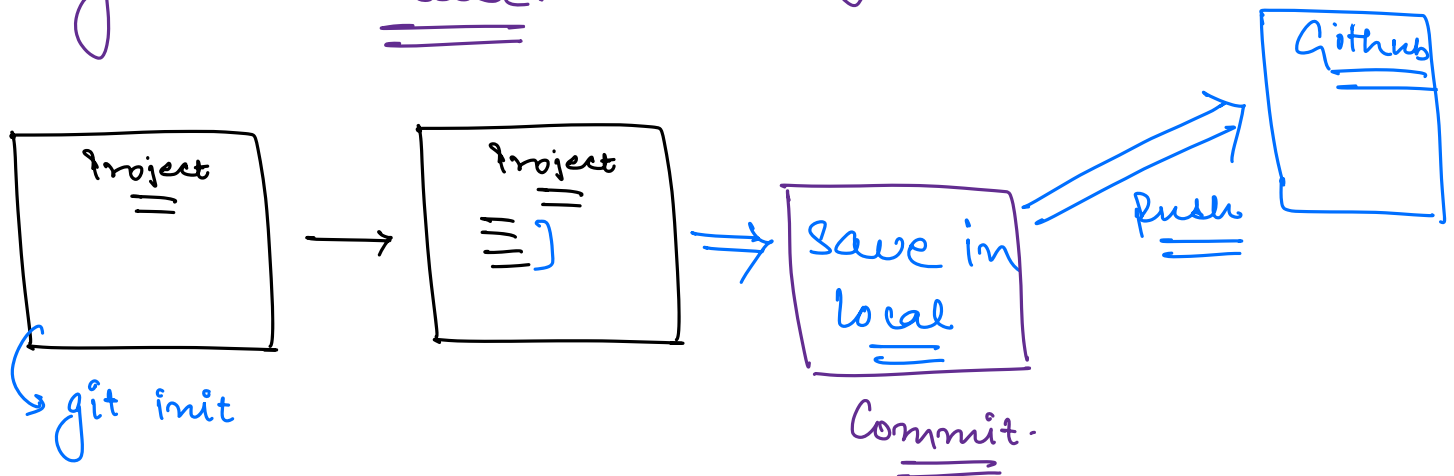
↳ Not VCS.



Git (Vs) Github

Software to manage versions of our codebase.

⇒ Server
Place to host git managed project.

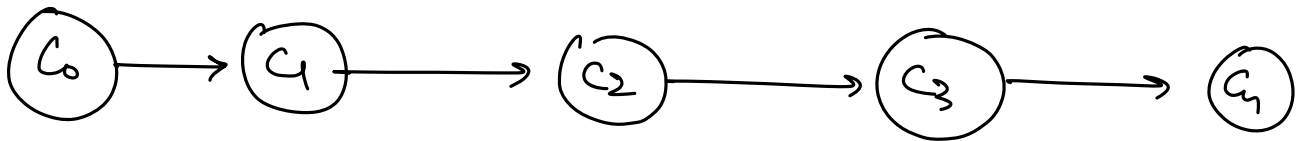
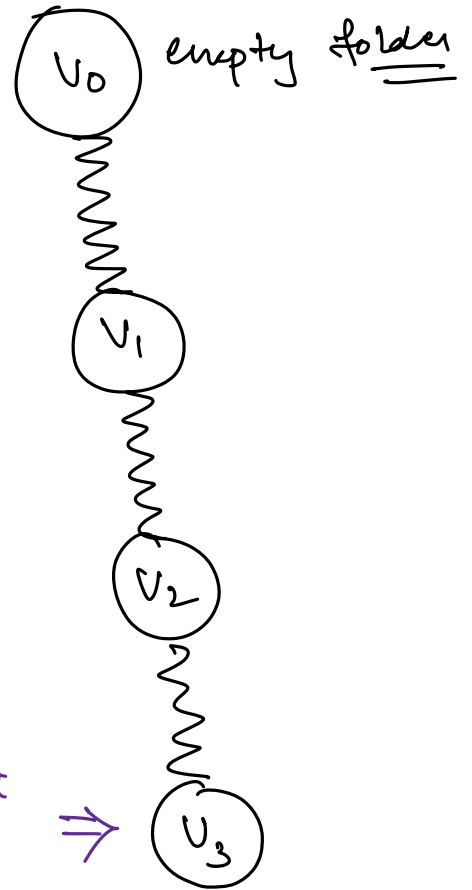


GIT Commit.

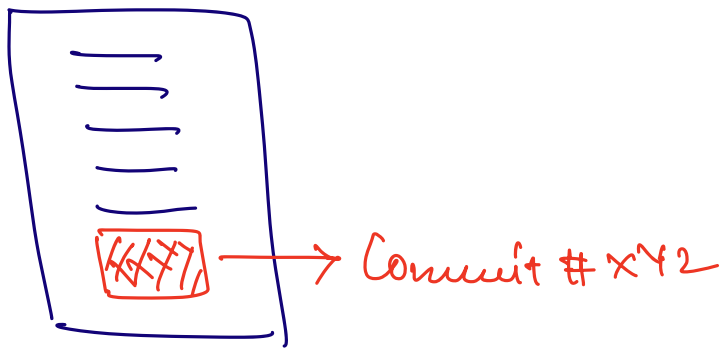


Command to save code
changes.

⇒ Every commit is nothing but
changes made on top of
the previous commit.



⇒ Every commit just stores the delta on
previous commit. diff.



Commit



globally
Unique Id

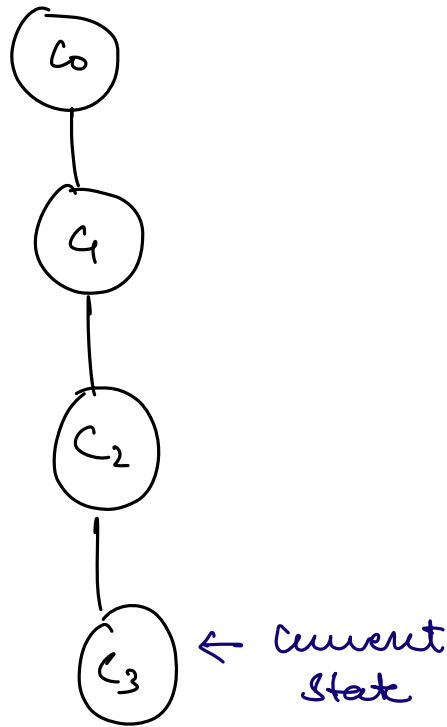
⇒ Every commit is IMMUTABLE.

⇒ Once a commit has been made, we can't edit any attr of it.

⇒ If we try to edit a commit, instead of editing, it creates a new commit.

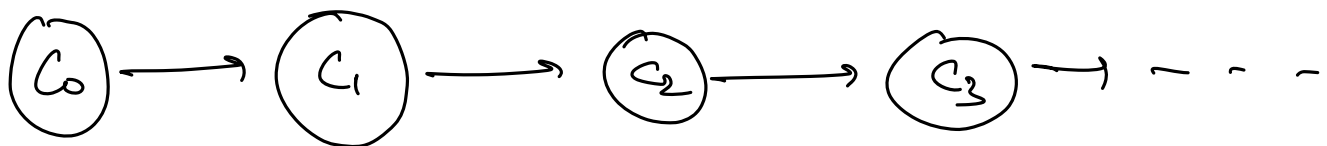
$\Rightarrow$ Once a commit is made, it can't be lost.

$\Rightarrow$ Git ensures no can play with the code history.



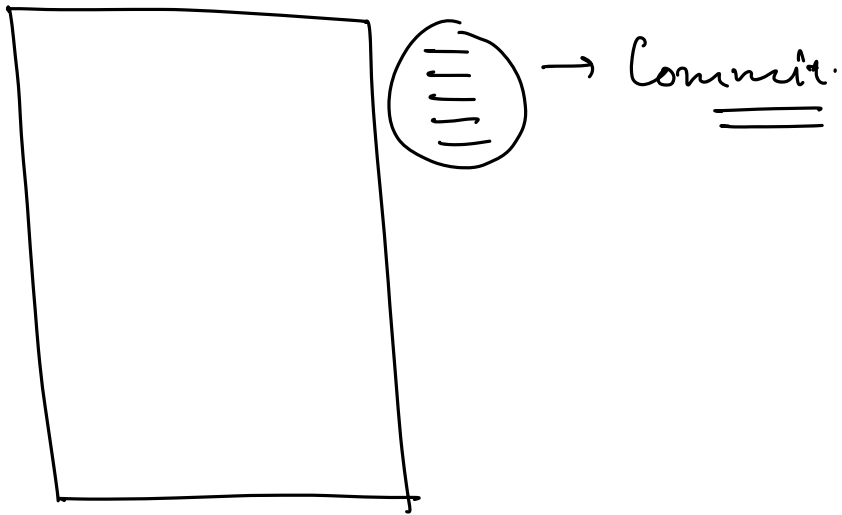
CommitID = hash(title, desc, author, timestamp, m/c info, list <previous commit IDs>)

How Git works?



Every commit just contains the delta over the previous commit.

⇒



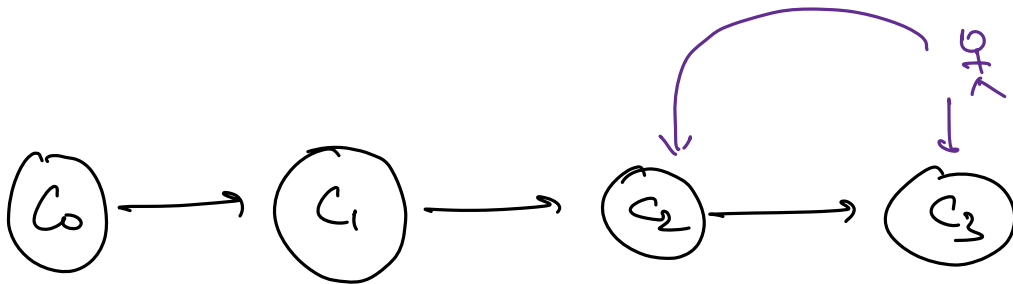
I) Install Git.

II) Create a project folder.

III) Git Init

IV) Make the change.

V) Save the changes.



————— * —————